Chapter 4

Reproducible Mass Spectrometry Project Architecture

Lucas VHH TRAN[1]

Abstract

Reproducibility in mass spectrometry requires deterministic tracking of software versions, operating system dependencies, parameter configurations, and intermediate file caches. Interactive graphical user interfaces obscure parameter histories and prevent exact replication across laboratories. In contrast, scripted pipelines define analytical operations as explicit code. A production-grade computational MS project relies on four architectural layers: portable file referencing that prevents hardcoded operating-system path failures, dependency freezing via package lockfiles (`renv.lock`), full-stack POSIX containerization (Docker), and directed acyclic graph (DAG) caching (`targets`). DAG caching tracks cryptographic content hashes of raw spectral inputs and parameter values, recalculating only targets invalidated by code or parameter mutations while skipping computationally expensive peak-picking steps. This chapter presents the architecture of portable project layouts, cryptographic checksum verification, lockfile dependency auditing, DAG caching dynamics, and runtime contract assertions in R 4.6.1.

Keywords: Reproducibility, renv lockfile, targets DAG, Cryptographic hash, Pipeline caching, Docker containerization, Runtime assertions.

Chapter Overview and Architectural Roadmap

Table 4.1 outlines the four tiers of the reproducibility stack for mass spectrometry research.

Methodological Note

Tutor Warning: The "Works on My Machine" Fallacy

An R script that executes successfully in an interactive R session can silently yield divergent results six months later if package versions or system shared libraries shift. An undocumented update to a peak detection package can alter peak boundary integration, changing quantitative matrices and downstream biological conclusions. Long-term reproducibility requires version-controlled lockfiles and automated

[1]CK-CARE Davos, Christine Kühne--Center for Allergy Research and Education, Davos, Switzerland. Email: lucas.tran@ck-care.ch

Table 4.1: Four tiers of the reproducible mass spectrometry computing architecture.

Tier	Tool / Specification	Mechanism	Failure Mode Addressed
1. Workspace	RStudio Project / `here`	Anchor relative paths to root `.Rproj` directory	Hardcoded path breakage (`C:/Users/`)
2. Packages	`renv` (`renv.lock`)	Freezes exact CRAN and Bioconductor commit versions	API deprecation and version drift
3. Operating System	Docker / Rocker image	Bundles C/C++ system libraries (`libnetcdf`)	Dynamic link failures across Linux/OS
4. Pipeline DAG	`targets`	Cryptographic hash tracking of data and parameters	Redundant re-computation and stale figures

pipeline graphs from the initial run.

4.1 Project Directory Topology and Path Portability

Computational mass spectrometry projects organize data files, transformation scripts, and reports into distinct directories. Mixing raw vendor files with intermediate CSV outputs risks accidental modification of original instrument data.

4.1.1 Standard Project Directory Layout

A production mass spectrometry project uses a standardized directory layout anchored by an RStudio Project file:

```
ms_project_root/
  |-- ms_project.Rproj     # Anchors project root directory
  |-- data/
  |   |-- raw/             # Read-only vendor mzML / raw files
  |   \-- processed/       # SummarizedExperiment / RDS outputs
  |-- R/                   # Modular analysis functions
  |-- reports/             # Quarto (.qmd) documents and PDF/HTML output
  |-- _targets.R           # Pipeline DAG definition
  |-- _targets/            # Automated target cache (git-ignored)
  |-- renv.lock            # Exact R package dependency manifest
  |-- Dockerfile           # POSIX container definition
  \-- .gitignore           # Excludes raw data and local cache
```

4.1.2 Path Portability via Anchor Files

Hardcoding absolute file paths (such as `"C:/Users/lucas/data/raw/run.mzML"`) breaks pipeline execution when a collaborator runs the code on a different computer or when the directory moves. The `here` package

locates the project root dynamically by identifying the directory containing `.Rproj` or `.git`. It constructs platform-independent paths using forward slashes:

$$\text{path} = \text{here}(\text{"data"}, \text{"raw"}, \text{"run_01.mzML"}) \tag{4.1}$$

Listing 4.1 computes MD5 and SHA-256 cryptographic hashes for an ingested raw spectral file.

R Lab Listing 1: Listing 4.1: Cryptographic Checksum Integrity Verification [R 4.4 / Bioconductor]

```
library(digest)

# Generate simulated raw mzML file in temporary workspace
temp_dir <- tempfile("ms_repro_audit_")
dir.create(temp_dir, recursive = TRUE)
raw_dir  <- file.path(temp_dir, "data", "raw")
dir.create(raw_dir, recursive = TRUE)

sim_raw_file <- file.path(raw_dir, "run_01.mzML")
writeLines(c("<?xml version='1.0' encoding='UTF-8'?>",
             "<mzML xmlns='http://psi.hupo.org/ms/mzml'>",
             "  <run id='sim_run_01' defaultInstrumentConfigurationRef='LCMS'>",
             "    <spectrum index='0' id='scan=1' defaultArrayLength='500'/>",
             "  </run>",
             "</mzML>"), con = sim_raw_file)

# Calculate MD5 and SHA-256 checksums
sha256_raw <- digest::digest(sim_raw_file, algo = "sha256", file = TRUE)
md5_raw    <- digest::digest(sim_raw_file, algo = "md5", file = TRUE)
bytes_raw  <- file.info(sim_raw_file)$size

cat(sprintf("Simulated Raw File: %s\n", basename(sim_raw_file)))
cat(sprintf("  File Size: %d bytes\n", bytes_raw))
cat(sprintf("  MD5 Checksum:    %s\n", md5_raw))
cat(sprintf("  SHA-256 Hash:    %s\n", sha256_raw))
```

```
Simulated Raw File: run_01.mzML
File Size: 233 bytes
MD5 Checksum:    10469785b424d205843712a94c037a76
SHA-256 Hash:    0e3f3fb807b1f1bad15147b3a8f356f4ba85962734d904a115ae3adfde352ace
```

The raw `run_01.mzML` file (233 bytes) produces SHA-256 hash `0e3f3fb8...`. If a byte changes during network transfer or disk storage, the cryptographic checksum changes immediately, exposing data corruption.

4.2 Software Environment Freezing: renv and Docker

R packages undergo continuous releases. Bioconductor coordinates releases every six months, while CRAN updates packages asynchronously. An analysis written against `xcms` version 3.20 may fail under version 4.0 if function arguments change.

4.2.1 Package Version Pinning with renv

The `renv` package establishes an isolated, project-local package library. The dependency graph is recorded in a JSON manifest (`renv.lock`). This file specifies the exact package versions, sources (CRAN, Bioconductor,

or GitHub), and git commit SHAs:

$$\mathbf{E} = \{(p_k, v_k, s_k, h_k)\}_{k=1}^{K} \tag{4.2}$$

where p_k is the package name, v_k is the semantic version string, s_k is the repository source, and h_k is the commit hash.

Listing 4.2 parses and audits the live `renv.lock` file from the monograph repository.

R Lab Listing 2: Listing 4.2: renv Lockfile Parsing and Architecture Audit [R 4.4 / Bioconductor]

```
library(jsonlite)

proj_lock <- "renv.lock"
lock_data <- jsonlite::fromJSON(proj_lock)
r_version <- lock_data$R$Version
bioc_ver  <- lock_data$Bioconductor$Version
pkg_names <- names(lock_data$Packages)
n_pkgs    <- length(pkg_names)

repos <- vapply(lock_data$Packages, function(x) {
  if (!is.null(x$Repository)) x$Repository else "Other/GitHub"
}, character(1))
repo_counts <- table(repos)

cat(sprintf("Project renv.lock Found: %s\n", normalizePath(proj_lock)))
cat(sprintf("  Recorded R Version:            %s\n", r_version))
cat(sprintf("  Recorded Bioconductor Version: %s\n", bioc_ver))
cat(sprintf("  Total Locked Packages:         %d\n", n_pkgs))
cat("  Repository Distribution:\n")
for (repo in names(repo_counts)) {
  cat(sprintf("    - %s: %d packages (%.1f%%)\n", repo, repo_counts[repo],
              repo_counts[repo] / n_pkgs * 100))
}
```

```
Project renv.lock Found: C:\Users\lucas\OneDrive\Statistics\RforMS\renv.lock
Recorded R Version:            4.6.1
Recorded Bioconductor Version: 3.23
Total Locked Packages:         159
Repository Distribution:
- Bioconductor 3.23: 9 packages (5.7%)
- CRAN: 111 packages (69.8%)
- https://bioc-release.r-universe.dev: 37 packages (23.3%)
- https://bioc.r-universe.dev: 1 packages (0.6%)
- Other/GitHub: 1 packages (0.6%)
```

The audit confirms 159 locked packages pinned to R 4.6.1 and Bioconductor 3.23. CRAN accounts for 111 packages (69.8%), Bioconductor repositories account for 47 packages (29.6%), and GitHub accounts for 1 package (0.6%).

4.2.2 Full-Stack POSIX Containerization (Docker)

The `renv.lock` manifest pins R code, but R packages that parse mass spectrometry data depend on underlying C and C++ system libraries (such as NetCDF, HDF5, and XML2). Divergent system libraries can cause segmentation faults or numerical differences across operating systems.

Docker encapsulates the operating system, system libraries, R runtime, and package library into an immutable image. A standard container definition for mass spectrometry analysis builds from a Rocker base image:

```
FROM rocker/tidyverse:4.4.1
RUN apt-get update && apt-get install -y \
    libnetcdf-dev \
    libhdf5-dev \
    libxml2-dev
COPY . /workspace
WORKDIR /workspace
RUN R -e "renv::restore()"
CMD ["R", "-e", "targets::tar_make()"]
```

Running this container reproduces the pipeline execution independently of host operating system configurations.

4.3 Directed Acyclic Graph (DAG) Pipeline Automation

Mass spectrometry pipelines involve computationally intensive stages: CentWave peak detection on 100 samples can require several hours. Re-running the entire pipeline after correcting a typographical error in a figure title is inefficient.

The `targets` package organizes an analysis into a Directed Acyclic Graph (DAG). Each step in the pipeline is defined as a target:

$$T_j = f_j(T_{j_1}, T_{j_2}, \ldots, \theta_j) \tag{4.3}$$

where T_j depends on upstream targets and parameter values θ_j.

4.3.1 Cryptographic Target Invalidation

The pipeline engine computes a cryptographic hash for each target:

$$H(T_j) = \text{hash}\left(H(T_{j_1}) \,\|\, H(T_{j_2}) \,\|\, \text{hash}(f_j) \,\|\, \text{hash}(\theta_j)\right) \tag{4.4}$$

When `tar_make()` executes:

1. If $H(T_j)$ matches the stored hash in the `_targets/` cache, the step is skipped and the cached object is loaded from disk in $O(1)$ time.
2. If an upstream input or parameter θ_j changes, $H(T_j)$ changes, triggering re-computation of T_j and all downstream dependent targets. Unrelated parallel branches remain untouched.

Listing 4.3 simulates DAG hash generation, zero-cost cache skipping, and parameter invalidation.

R Lab Listing 3: Listing 4.3: DAG Pipeline Dependency Caching Simulation [R 4.4 / Bioconductor]

```r
library(digest)

# Step 1: Compute initial DAG hashes for a 3-step MS pipeline
ppm_param <- 15.0
cache_store <- list()
```

```
hash_target1 <- digest::digest(list("raw_mzml_data"), algo = "xxhash64")
hash_target2 <- digest::digest(list(hash_target1, ppm = ppm_param), algo = "xxhash64")
hash_target3 <- digest::digest(list(hash_target2, method = "max_intensity"), algo = "xxhash64")

cat("Initial DAG Hash Tree Generation:\n")
cat(sprintf("  Target 1 [raw_file]:       %s\n", hash_target1))
cat(sprintf("  Target 2 [peak_detection]: %s (ppm = %.1f)\n", hash_target2, ppm_param))
cat(sprintf("  Target 3 [quant_matrix]:   %s\n", hash_target3))

# Populate cache
cache_store[[hash_target1]] <- list(status = "SKIPPED")
cache_store[[hash_target2]] <- list(status = "SKIPPED")
cache_store[[hash_target3]] <- list(status = "SKIPPED")

# Invocation 2: Zero input changes -> 100% cache hit
t_start <- Sys.time()
inv2_recompute <- c(
  t1 = is.null(cache_store[[hash_target1]]),
  t2 = is.null(cache_store[[hash_target2]]),
  t3 = is.null(cache_store[[hash_target3]])
)
t_inv2 <- as.numeric(difftime(Sys.time(), t_start, units = "secs"))

cat("\nInvocation 2 (Zero Input Changes):\n")
cat(sprintf("  Recomputed Targets: %d / 3 (Targets Skipped: %d)\n",
            sum(inv2_recompute), sum(!inv2_recompute)))
cat(sprintf("  Pipeline Verification Time: %.6f seconds (0 ms recompute penalty)\n", t_inv2))

# Invocation 3: Parameter mutation: ppm 15.0 -> 10.0
ppm_param_new <- 10.0
hash_target2_new <- digest::digest(list(hash_target1, ppm = ppm_param_new), algo = "xxhash64")
hash_target3_new <- digest::digest(list(hash_target2_new, method = "max_intensity"), algo = "xxhash64")

inv3_recompute <- c(
  t1_raw_file       = is.null(cache_store[[hash_target1]]),     # Cached!
  t2_peak_detection = is.null(cache_store[[hash_target2_new]]), # Invalidated!
  t3_quant_matrix   = is.null(cache_store[[hash_target3_new]])  # Downstream cascade!
)

cat("\nInvocation 3 (Parameter Mutation: ppm 15.0 -> 10.0):\n")
cat(sprintf("  Target 1 [raw_file]:       %s\n", ifelse(inv3_recompute["t1_raw_file"], "RECOMPUTED", "SKIPPED
    (Cached)")))
cat(sprintf("  Target 2 [peak_detection]: %s\n", ifelse(inv3_recompute["t2_peak_detection"], "RECOMPUTED
    (Invalidated)", "SKIPPED")))
cat(sprintf("  Target 3 [quant_matrix]:   %s\n", ifelse(inv3_recompute["t3_quant_matrix"], "RECOMPUTED
    (Downstream cascade)", "SKIPPED")))
cat(sprintf("  Total Recomputations: %d / 3 targets\n", sum(inv3_recompute)))
```

```
Initial DAG Hash Tree Generation:
Target 1 [raw_file]:      7ef71c205e4d1ac5
Target 2 [peak_detection]: aa6b898bffea970e (ppm = 15.0)
Target 3 [quant_matrix]:  dd99504415ea3986

Invocation 2 (Zero Input Changes):
Recomputed Targets: 0 / 3 (Targets Skipped: 3)
```

```
Pipeline Verification Time: 0.000726 seconds (0 ms recompute penalty)
Invocation 3 (Parameter Mutation: ppm 15.0 -> 10.0):
Target 1 [raw_file]:      SKIPPED (Cached)
Target 2 [peak_detection]: RECOMPUTED (Invalidated)
Target 3 [quant_matrix]:  RECOMPUTED (Downstream cascade)
Total Recomputations: 2 / 3 targets
```

In Invocation 2, verification completes in 0.000726 seconds with zero recomputations. In Invocation 3, mutating the mass tolerance from 15.0 to 10.0 ppm invalidates Target 2 and cascades to Target 3, while preserving the raw file ingestion target (Target 1).

4.4 Defensive Runtime Contract Testing on MS Assay Matrices

Silent data corruption occurs when erroneous preprocessing produces negative intensities, NaN values, or unexpected dimension drops without halting execution. Runtime contract assertions using `testthat` verify mathematical invariants before data enter statistical models.

Listing 4.4 tests three structural contracts on a simulated feature abundance matrix: dimension consistency, intensity non-negativity, and missingness rate bounds.

R Lab Listing 4: Listing 4.4: Unit Testing and Defensive Contract Auditing [R 4.4 / Bioconductor]

```r
library(testthat)

set.seed(99)
test_assay <- matrix(abs(rnorm(100, mean = 5000, sd = 1000)), nrow = 20, ncol = 5)
colnames(test_assay) <- paste0("Sample_", 1:5)
rownames(test_assay) <- paste0("Feat_", 1:20)
test_assay[1, 2] <- NA_real_

# Execute contract assertions
test_results <- testthat::test_that("MS Assay Integrity Contracts", {
  # Contract 1: Dimensional invariance
  testthat::expect_equal(dim(test_assay), c(20L, 5L))
  # Contract 2: Intensity non-negativity
  testthat::expect_true(all(test_assay[!is.na(test_assay)] >= 0))
  # Contract 3: Missingness upper bound (< 10%)
  missing_pct <- mean(is.na(test_assay)) * 100
  testthat::expect_lt(missing_pct, 10.0)
})

cat("Testthat Contract Suite Execution: 3 / 3 assertions passed.\n")
cat(sprintf("  Matrix Dimensions: %d features x %d samples\n", nrow(test_assay), ncol(test_assay)))
cat(sprintf("  Minimum Valid Intensity: %.2f counts\n", min(test_assay, na.rm = TRUE)))
cat(sprintf("  Observed Missingness Rate: %.2f%% (Contract threshold: < 10.00%%)\n",
            mean(is.na(test_assay)) * 100))
```

```
Testthat Contract Suite Execution: 3 / 3 assertions passed.
Matrix Dimensions: 20 features x 5 samples
Minimum Valid Intensity: 1959.07 counts
Observed Missingness Rate: 1.00% (Contract threshold: < 10.00%)
```

All three contracts pass: the matrix contains 20 features and 5 samples, the minimum observed intensity

is 1,959.07 counts, and the observed missingness rate is 1.00%, satisfying the $< 10.00\%$ contract threshold.

4.5 Worked Exercises

Exercise 4.1: Portable Path Resolution with `here::here()`

Problem Statement: Explain why hardcoding paths such as `"C:/Users/name/data/raw/sample.mzML"` causes pipeline failures across collaborators' systems and explain how `here::here()` eliminates this failure mode.
Solution: Hardcoded paths contain user-specific root directories (`"C:/Users/name/"`) and Windows backslash separators that do not exist on other machines or POSIX operating systems (Linux/macOS). When another user runs the script, the file cannot be located, halting execution.

The function `here::here("data", "raw", "sample.mzML")` dynamically detects the project root by searching upward from the current working directory for an anchor file (such as `.Rproj` or `.git`). It joins directory components using the host operating system's native file separator. Moving the project directory to another disk, computer, or cloud container preserves valid path resolution without altering code.

Exercise 4.2: Dependency Tier Scope: `renv` vs Docker

Problem Statement: Distinguish the scope of reproducibility provided by `renv` from that provided by Docker. State a failure scenario where `renv.lock` is insufficient to guarantee identical numerical outputs.
Solution:

1. **`renv`**: Freezes the R package layer. It records exact package version strings, repository origins, and GitHub commit SHAs in `renv.lock`, ensuring that identical R bytecode is loaded.

2. **Docker**: Freezes the complete POSIX operating system layer. It bundles the Linux distribution kernel interface, compilers, system libraries (`libnetcdf`, `libxml2`, `libhdf5`), and BLAS/LAPACK linear algebra implementations.

Failure Scenario: An analyst runs `xcms` on Linux with NetCDF version 4.8, while a collaborator restores the identical `renv.lock` on macOS with NetCDF version 4.9. If NetCDF 4.9 introduces a slight difference in floating-point decompression of vendor raw arrays, downstream peak integration boundaries will diverge, producing altered quantitative values despite identical R package versions. Docker eliminates this divergence by freezing the compiled C libraries.

Exercise 4.3: DAG Invalidation Dynamics in `targets`

Problem Statement: In a mass spectrometry `targets` pipeline comprising:

1. `raw_files`: Reads list of 50 mzML files.

2. `ppm_tolerance`: Target defined as numeric scalar 15.0.

3. `chrom_peaks`: Calls `findChromPeaks(raw_files, ppm = ppm_tolerance)`.

4. `quant_matrix`: Extracts feature intensity values from `chrom_peaks`.

5. `report`: Renders Quarto report displaying the volcano plot.

If the analyst alters `ppm_tolerance` from 15.0 to 10.0, identify which targets `tar_make()` skips and which targets it recomputes.
Solution:

- `raw_files`: **SKIPPED (Cached)**. The raw mzML files and their file-level checksums are unmodified; their cryptographic hash is identical.
- `ppm_tolerance`: **RECOMPUTED**. The target definition mutated from 15.0 to 10.0.
- `chrom_peaks`: **RECOMPUTED**. This target depends on `ppm_tolerance`; the change in upstream hash invalidates the cached peak table.
- `quant_matrix`: **RECOMPUTED**. Depends on the newly recomputed `chrom_peaks`.
- `report`: **RECOMPUTED**. Depends on `quant_matrix`; the Quarto report is re-rendered with updated quantification results.

Exercise 4.4: High-Performance Serialization Formats

Problem Statement: Explain why default R serialization via `saveRDS()` creates bottlenecks in large mass spectrometry pipelines and state two alternative serialization formats supported by `targets`.
Solution: Default `saveRDS()` uses single-threaded gzip compression. For large mass spectrometry objects ($>$ 2 GB), gzip compression and decompression require substantial CPU time and inflate RAM during serialization.

Two high-performance alternatives are:

1. `qs` **(Quick Serialization)**: Uses multi-threaded Zstandard (zstd) compression, providing up to 5$\times$ faster read and write throughput than `saveRDS()` with comparable compression ratios.
2. `fst` **(Fast Serialization of DataFrames)**: Provides multi-threaded, memory-mapped random-access storage for tabular feature matrices and PSM tables, enabling instant decompression of specific column subsets without loading the entire matrix into RAM.

Bibliography

[1] Landau, W.M. The targets R package: a dynamic Make-like function-oriented pipeline toolkit for reproducible workflows. *Journal of Open Source Software*, 6(57):2959, 2021. https://doi.org/10.21105/joss.02959.

[2] Ushey, K., Wickham, H. *renv: Project Environments for R*. R package version 1.0.7, 2024. https://CRAN.R-project.org/package=renv.

[3] Knuth, D.E. Literate programming. *The Computer Journal*, 27(2):97--111, 1984. https://doi.org/10.1093/comjnl/27.2.97.

[4] Wickham, H. testthat: Get started with testing. *The R Journal*, 3(1):5--10, 2011. https://doi.org/10.32614/RJ-2011-002.

[5] Müller, K. *here: A Simpler Way to Find Your Files*. R package version 1.0.1, 2020. https://CRAN.R-project.org/package=here.

[6] Boettiger, C. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71--79, 2015. https://doi.org/10.1145/2723872.2723882.